

Cloud Architecture

Assignment-2 (Report)

Submitted By: Fahamidul Hasan (20115491)

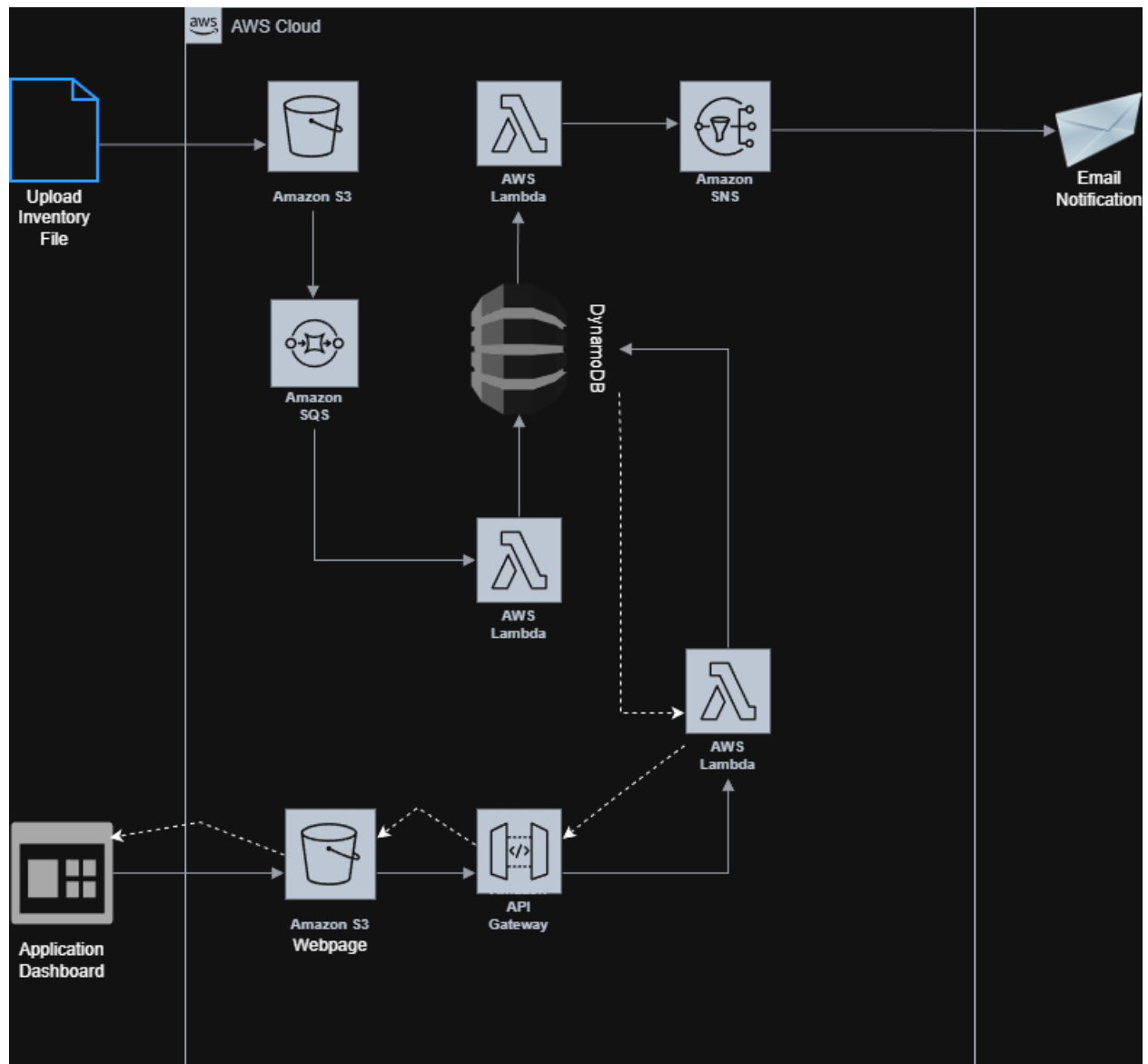
Contents

Application Scenario	2
Architecture Diagram	2
Workflow	3
AWS Well Architected Framework	8
Conclusion	9
Plagiarism Declaration	9

Application Scenario

The scenario is based on a library inventory system for which a serverless AWS solution is proposed. The library may have multiple branches in different cities, in case any branch is out of stock for a specific book, an inventory alert will be sent to the designated email. The inventory details of all branches can also be viewed through a web page.

Architecture Diagram



The architecture diagram lists all AWS services used for this serverless solution and visualizes their workflow. In this architecture I have used Amazon S3, Amazon SQS, AWS Lambda, DynamoDB, Amazon SNS, API Gateway to build a serverless loosely coupled system.

Workflow

- ⇒ Users upload branch specific CSV files to Amazon S3 bucket. The S3 Bucket has an event notification set to S3toSQS queue on all objects create events.

library-inventory-20115491 [Info](#)

[Objects](#) | [Metadata](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

Objects (3) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	Dublin.csv	csv	November 22, 2025, 20:30:04 (UTC+00:00)	88.0 B	Standard
☐	Galway.csv	csv	November 23, 2025, 14:55:15 (UTC+00:00)	93.0 B	Standard
☐	Waterford.csv	csv	November 23, 2025, 16:44:26 (UTC+00:00)	101.0 B	Standard

Event notifications (1) [Edit](#) [Delete](#) [Create event notification](#)

Send a notification when specific events occur in your bucket. [Learn more](#)

☐	Name	Event types	Filters	Destination type	Destination
☐	S3toSQS	All object create events	-	SQS queue	S3UploadQueue

Amazon EventBridge [Edit](#)

For additional capabilities, use Amazon EventBridge to build event-driven applications at scale using S3 event notifications. [Learn more](#) or [see EventBridge pricing](#)

Send notifications to Amazon EventBridge for all events in this bucket
Off

- ⇒ SQS stores the messages in a Queue. This ensures that the architecture is loosely coupled as the producer (S3) and the consumer (Lambda) now have a buffer (SQS) between them.

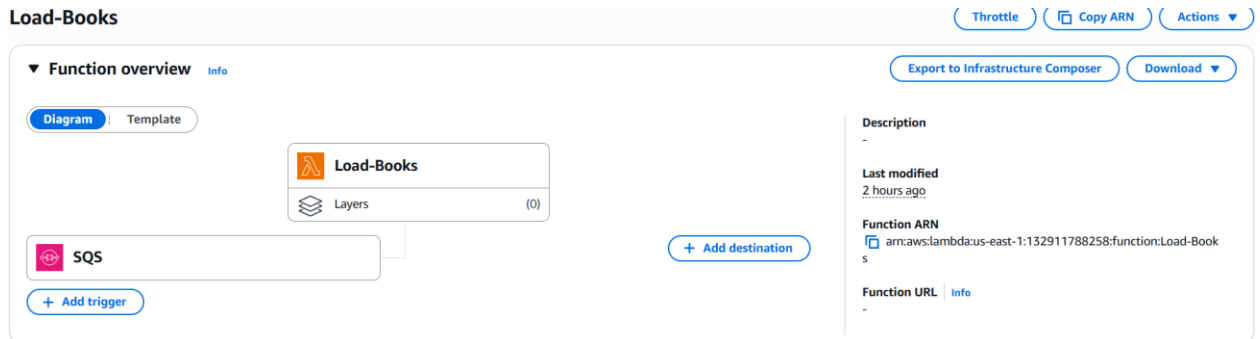
S3UploadQueue [Edit](#) [Delete](#) [Purge](#) [Send and receive messages](#) [Start DLQ redrive](#)

Details [Info](#)

Name S3UploadQueue	Type Standard	ARN arn:aws:sqs:us-east-1:132911788258:S3UploadQueue
Encryption Amazon SQS key (SSE-SQS)	URL https://sqs.us-east-1.amazonaws.com/132911788258/S3UploadQueue	Dead-letter queue -

[More](#)

- ⇒ Lambda is triggered by SQS. It picks up the batch of messages and processes them and finally uploads them to DynamoDB.



```
try:
    with open(local_filename) as csvfile:
        reader = csv.DictReader(csvfile, delimiter=',')
        for row in reader:
            print("CSV Row:", row)

            libraryTable.put_item(
                Item={
                    'Library': row['library'],
                    'Book': row['book'],
                    'Count': int(row['count'])
                }
            )
            total_records += 1
```

The code snippet shows the phase of writing new data to DynamoDB table.

- ⇒ Open uploaded CSV file
- ⇒ Read each row
- ⇒ print the entire row to CloudWatch
- ⇒ Insert each row to DynamoDB table
- ⇒ Increment record counter that can later be printed to CloudWatch log

⇒ DynamoDB table storing the records

Table: LibraryBooks - Items returned (9) Actions Create item

Scan started on November 27, 2025, 14:53:51

< 1 > ⚙

☐	Library (String) ▾	Book (String) ▾	Count ▾
☐	Dublin	Harry Potter	23
☐	Dublin	The Hobbit	10
☐	Dublin	The Witcher	0
☐	Galway	A Song of Ice and Fire	0
☐	Galway	Hobbit	5
☐	Galway	The Witcher	10
☐	Waterford	A Song of Ice and Fire	0
☐	Waterford	Hobbit	5
☐	Waterford	The Witcher	0

⇒ DynamoDB then triggers another Lambda function

Check-Books Throttle Copy ARN Actions

Export to Infrastructure Composer Download

▼ **Function overview** Info

Diagram Template

 **Check-Books**

 Layers (0)

 **DynamoDB**

+ Add trigger + Add destination

Description
-

Last modified
2 hours ago

Function ARN
 arn:aws:lambda:us-east-1:132911788258:function:Check-Books

Function URL Info
-

```
for record in event['Records']:
    newImage = record['dynamodb'].get('NewImage', {})
    count = int(newImage['Count']['N'])

    if count == 0:
        library = newImage['Library']['S']
        book = newImage['Book']['S']

        message = f"OUT OF STOCK: '{book}' at {library} library."
        print(message)

        sns.publish(
            TopicArn=topic_arn,
            Message=message,
            Subject="Inventory Alert!"
        )
```

- ⇒ The code snippet shows the phase of publishing an SNS notification based on if any row contains count = 0
- => Read a new row
- => Convert value of count to int [Default is Number: N]
- => If count = 0 then publish that the book is out of stock at that library through SNS

LibraryOutOfStock

[Edit](#) [Delete](#) [Publish message](#)

Details

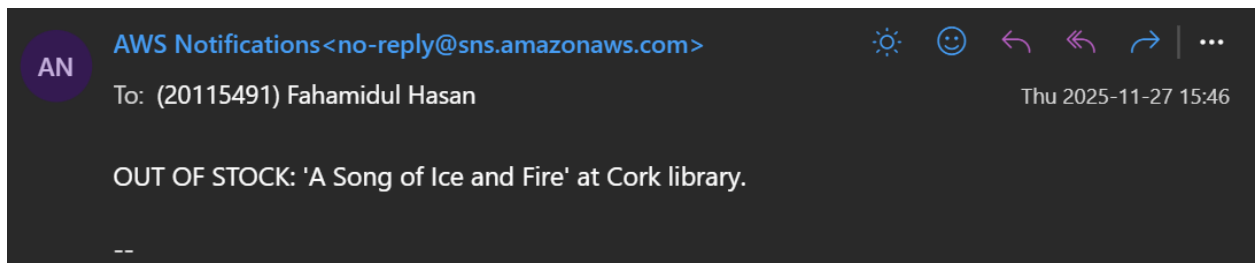
Name LibraryOutOfStock	ARN arn:aws:sns:us-east-1:132911788258:LibraryOutOfStock	Display name -	Type Standard
Topic owner 132911788258			

[<](#)
[Subscriptions](#)
[Access policy](#)
[Data protection policy](#)
[Delivery policy \(HTTP/S\)](#)
[Delivery status logging](#)
[Encryption](#)
[Tags](#)
[>](#)

Subscriptions (1)
[Edit](#)
[Delete](#)
[Request confirmation](#)
[Confirm subscription](#)
[Create subscription](#)

ID	Endpoint	Status	Protocol
5168889c-ff20-4b05-87b3-b4e093...	20115491@setu.ie	Confirmed	EMAIL

An email is then sent through the SNS to the designated email address as Inventory alert.



⇒ To visualize the inventory, a static web page is hosted in an S3 Bucket.

library-web-20115491 [Info](#)

[Objects](#) [Metadata](#) [Properties](#) [Permissions](#) [Metrics](#) [Management](#) [Access Points](#)

Objects (1) [Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	index.html	html	November 23, 2025, 16:51:09 (UTC+00:00)	2.2 KB	Standard

→ Not secure library-web-20115491.s3-website-us-east-1.amazonaws.com

Google Gemini ChatGPT YouTube SETU Moodle Net Ninja LeetCode - The Wor... Launch AWS Acade... Home - Chess.com Stakeholder Analysi... CPC Project Octago...

Library Inventory Viewer

Select Library: [All Libraries](#) [Load Books](#)

Library	Book	Count
Cork	A Song of Ice and Fire	0
Cork	Hobbit	5
Cork	The Witcher	11
Dublin	Harry Potter	23
Dublin	The Hobbit	10
Dublin	The Witcher	0
Galway	A Song of Ice and Fire	0
Galway	Hobbit	5
Galway	The Witcher	10
Waterford	A Song of Ice and Fire	0
Waterford	Hobbit	5
Waterford	The Witcher	0

To retrieve the data from DynamoDB database, API Gateway is used.

APIs (1/1) [Refresh](#) [Delete](#) [Create API](#)

	Name	Description	ID	Protocol	API endpoint type	Created	Security policy	API status
☐	LibraryAPI		kopjimpuyysg	REST	Regional	2025-11-23	SecurityPolicy_TLS13_1_3_2025_09	Available

The API Gateway triggers another Lambda function that fetches the data from DynamoDB table.

GetLibraryBooks [Throttle](#) [Copy ARN](#) [Actions](#)

[Export to Infrastructure Composer](#) [Download](#)

Function overview [Info](#)

[Diagram](#) [Template](#)

GetLibraryBooks

Layers (0)

API Gateway

[+ Add trigger](#)

[+ Add destination](#)

Description

-

Last modified
3 hours ago

Function ARN
[arn:aws:lambda:us-east-1:132911788258:function:GetLibraryBooks](#)

Function URL [Info](#)

-

```
lambda_function.py X
lambda_function.py
8 def lambda_handler(event, context):
9     print("Event:", json.dumps(event))
10
11     params = event.get("queryStringParameters") or {}
12     library = params.get("library") if params else None
13
14     if library:
15         response = table.query(
16             KeyConditionExpression=Key('Library').eq(library)
17         )
18         items = response.get('Items', [])
19     else:
20         response = table.scan()
21         items = response.get('Items', [])
22
23     return {
24         "statusCode": 200,
25         "headers": {
26             "Content-Type": "application/json",
27             "Access-Control-Allow-Origin": "*",
28         },
29         "body": json.dumps(items, default=int)
30     }
31
```

The code snippet displays the phase of scanning for items in the database and returning the data back to the front end.

- => Read query parameters
- => If library name is provided then query in that library for books
- => Else query for all books
- => Return the data back to front end as JSON

AWS Well Architected Framework

Evaluation of the system based on WAF:

- ⇒ **Operational Excellence:** The system uses serverless components with CloudWatch logging to make it easy to monitor and troubleshoot.
- ⇒ **Security:** Access to DynamoDB is controlled, which makes the system secure.
- ⇒ **Reliability:** The SQS buffer ensures that no messages are lost in case lambda fails ensuring reliability.
- ⇒ **Performance Efficiency:** Serverless components scale automatically according to performance needs.
- ⇒ **Cost Optimization:** Serverless components are charged according to usage, optimizing cost.
- ⇒ **Sustainability:** Using serverless resources on demand has minimal environmental impact.

Conclusion

In conclusion, I have designed a cloud solution that aligns with the well architected framework, and I have also used serverless components to build a loosely coupled system.

Plagiarism Declaration

I certify that this assignment is all my own work and contains no plagiarism. By submitting this assignment, I agree to the following terms: Any text, diagrams or other material copied from other sources (including, but not limited to, books, journals and the internet) have been clearly acknowledged and referenced as such in the text by the use of 'quotation marks' (or indented italics for longer quotations) followed by the author's name and date [e.g. (Byrne, 2018)] either in the text or in a footnote/endnote. These details are then confirmed by a fuller reference in the bibliography. I have read Appendix 1 of the SETU Student Academic Misconduct Policy and Disciplinary Procedure, and I understand that only assignments which are free of plagiarism will be awarded marks. I further understand that the disciplinary procedure can lead to the suspension or permanent expulsion of students in serious cases.

- Fahamidul Hasan (20115491)